The server-side code that reads these values would also have to change to read from the query parameter rather than the request headers.

REDUCING HTTP METHODS

Another way to reduce preflight requests is to reduce the types of HTTP methods used. If your API supports `PUT` or `DELETE` requests, these requests must have a preflight request. There is no way to work around a preflight on `PUT` and `DELETE` requests.

But if the request were a `POST` instead of a `PUT`, it wouldn't need a preflight request (so long as it didn't also have custom headers). Certain protocols, such as JSON-RPC, are built entirely around `GET` and `POST` requests only. (JSON-RPC also gives excellent preflight caching because all requests are made to a single endpoint.) If avoiding preflight requests is important, you may want to investigate an alternative protocol.

This section covered strategies for handling preflight requests. It started by discussing common methods and headers that should be whitelisted. It then described a technique to give more fine-grained control over what types of request headers are allowed (a technique that can be expanded to allow any request header).

Next it discussed strategies for reducing preflight requests. Because preflight requests issue an additional HTTP request, reducing them can lead to performance gains, especially on resource-constrained devices such as mobile. A preflight request is issued whenever a request uses methods other than `GET` or `POST`, or when it uses custom headers. Reducing the need for either reduces the need for preflight requests.

This discussion of preflight requests rounds out the basic building blocks of a CORS server. But there are still a few more odds and ends to CORS requests. The next sections look into exposing headers to clients, and handling HTTP redirects.

6.7 Exposing response headers

The last remaining CORS feature is the ability to expose response headers to the client. Chapter 5 described how the Access-Control-Expose-Headers header could be used to indicate which, if any, response headers are visible to the client. It's not a critical CORS header in that the request will still succeed if this header isn't present, but it's useful if clients need to read response headers. If the server wants to give the client the ability to read the X-Powered-By response header, it would set the following header on the response:

```
Access-Control-Expose-Headers: X-Powered-By
```

Using the technique similar to the Access-Control-Allow-Headers header, the sample code in the following listing adds an option to set the Access-Control-Expose-Headers header.

Listing 6.12 Exposing response headers to the client

```
var corsOptions = {
  allowOrigin: createWhitelistValidator(originWhitelist),
  allowCredentials: true,
  shortCircuit: true,
```